UNIVERSIDADE DE BRASÍLIA

GRUPO PERLMAN - T03



MOTIVAFIT

DOCUMENTO DE ARQUITETURA

VERSÃO 1.7 — REVISADA

Brasília
2026

Bernardo Campos Caxito Silveira
Bruno Cruvinel Barros da Silva
Lucas Venâncio Pereira
Luís Felipe Pontes Vieira Andrade
Mateus Fernandes Dantas
Matheus Estevam Ferreira
Rafael Souto Lopes Laube

MOTIVAFIT

DOCUMENTO DE ARQUITETURA

VERSÃO 1.7 — REVISADA

Documento de arquitetura do produto MotivaFit, elaborado pelo Grupo Perlman – T03, Universidade de Brasília, na disciplina de Métodos de Desenvolvimento de Software (MDS).

Brasília
2026

Tabela 1 – Integrantes do grupo

Matrícula	Nome	Função	Pontos
232024966	Bernardo Campos Caxito Silveira	GQM	14,28
251039442	Bruno Cruvinel Barros da Silva	Documentação	14,29
252004290	Lucas Venâncio Pereira	Back-end	14,28
252023741	Luís Felipe Pontes Vieira Andrade	Back-end	14,28
251026248	Mateus Fernandes Dantas	Front-end	14,29
251013651	Matheus Estevam Ferreira	Back-end	14,29
211062428	Rafael Souto Lopes Laube	Front-end	14,29
Total de pontos (devem fechar 100 pontos para toda a equipe): 100,00			

Fonte: elaborado pelos autores (2026).

GQM: Goal-Question-Metric (objetivo–questão–métrica).

Tabela 2 – Histórico de revisões

Data	Versão	Descrição	Autor(es)
03/10/2026	1.0	Elaboração inicial do Documento de Arquitetura	Equipe Perlman
03/10/2026	1.1	Adição de subseções de metas e restrições detalhadas	Equipe Perlman
03/10/2026	1.2	Remoção de referências ao GitHub Actions (CI/CD)	Equipe Perlman
03/10/2026	1.3	Descrições do Documento de Visão	Equipe Perlman
03/10/2026	1.4	Correção das representações visuais dos diagramas UML	Equipe Perlman
03/10/2026	1.5	Integração completa do texto base e explicação dos fluxos dos diagramas	Equipe Perlman
03/10/2026	1.6	Revisão geral e alinhamento ao Documento de Visão 1.0.0: reintrodução do GitHub Actions/CI e da meta de cobertura de 70% (M3); login com Google como opcional pós-MVP; correção da numeração das seções e remoção de trechos de modelo e de duplicidades; redesenho dos diagramas (inclusão de desafios, grupos, feed, convites e catálogo de exercícios); inclusão de definições, tecnologias, contrato da API, segurança, rastreabilidade RF–módulos, CI/CD, riscos e bibliografia completa	Equipe Perlman
07/10/2026	1.7	Cópia revisada: prioridades alinhadas à Visão 1.0.1; modelos e oito diagramas corrigidos; recordes automáticos, histórico, social, segurança, CI, métricas e operação revisados.	Revisão documental solicitada

Fonte: elaborado pelos autores (2026).

SUMÁRIO

1 INTRODUÇÃO....3
1.1 Propósito....3
1.2 Escopo....3
1.3 Definições, acrônimos e abreviações....3
1.4 Documentos de referência....4
1.5 Organização do documento....4
2 REPRESENTAÇÃO ARQUITETURAL....5
2.1 Definições....5
2.2 Justificativa da escolha....5
2.2.1 Tecnologias e decisões arquiteturais....5
2.3 Detalhamento....6
2.3.1 Explicação do diagrama em camadas....8
2.3.2 Contrato da API REST....8
2.3.3 Mecanismos de segurança....9
2.4 Metas e restrições arquiteturais....10
2.4.1 Metas arquiteturais....10
2.4.1.1 Garantir modularidade e baixo acoplamento....10
2.4.1.2 Garantir qualidade contínua....10
2.4.1.3 Garantir segurança e controle de acesso....10
2.4.1.4 Garantir usabilidade e fluidez....11
2.4.1.5 Garantir evolução e extensibilidade....11
2.4.2 Restrições arquiteturais....11
2.4.2.1 Restrição tecnológica....12
2.4.2.2 Restrição de estilo arquitetural....12
2.4.2.3 Restrição de padronização de código....12
2.4.2.4 Restrição de versionamento e integração....12
2.4.2.5 Restrição operacional e acadêmica....12
2.5 Visões....12
2.5.1 Visão de uso....13
2.5.1.1 Diagrama de casos de uso....13
2.5.1.2 Diagrama de atividades....14
2.5.2 Visão de organização lógica....17
2.5.3 Visão estrutural....18
2.5.3.1 Modelo de dados relacional....19
2.5.3.2 Diagrama de componentes....22
2.6 Visão de Implantação....22
2.6.1 Estratégia e execução de testes....23
2.6.2 Integração contínua....24
2.7 Operação e ciclo de vida dos dados....24
2.8 Rastreabilidade com o Documento de Visão....25
2.9 Riscos arquiteturais e decisões pendentes....26
3 BIBLIOGRAFIA....28

1 INTRODUÇÃO

1.1 Propósito

Este documento descreve a arquitetura do sistema MotivaFit, desenvolvido pelo Grupo Perlman – T03 na disciplina de MDS – Métodos de Desenvolvimento de Software, edição do semestre de 2026. Seu propósito é fornecer uma visão abrangente e padronizada do sistema para desenvolvedores, testadores e demais interessados nos aspectos relacionados às tecnologias e às estruturas utilizadas no desenvolvimento do software, registrando as decisões arquiteturais e suas justificativas.

1.2 Escopo

O escopo está alinhado à cópia revisada do Documento de Visão, versão 1.0.1, incluída nesta revisão. O MotivaFit é uma aplicação web para organização de treinos, registro de execução e acompanhamento de progresso, com recursos posteriores de conquistas, desafios, rede social e painel administrativo. Hexad é referência de projeto para discutir motivações; não há classificação de usuários, questionário, adaptação por tipo nem regras executáveis baseadas em Hexad.

O MVP será concluído ao final da Sprint 4 e reúne exclusivamente os requisitos Must RF-01 a RF-11. RF-12, RF-18, RF-19, RF-20 e RF-22 passam de Must a Should, mantendo as Sprints 5, 7, 7, 8 e 9, respectivamente. Todos os requisitos fora do MVP são Should ou Could; RF-24 continua opcional nas Sprints 9 e 10. As Tabelas 7 e 13 e o backlog da cópia do Documento de Visão registram essa reclassificação. Não há antecipação de escopo obrigatório nem mudança das Sprints.

Não fazem parte do escopo desta arquitetura: aplicativos móveis nativos (o produto é uma aplicação web que deve se adaptar a telas de celular e de computador) e a criação de exercícios pelo usuário, pois os exercícios são selecionados a partir de uma tabela de exercícios predefinidos (RF-07).

1.3 Definições, acrônimos e abreviações

Tabela 3 – Definições, acrônimos e abreviações

Termo	Significado
API	Application Programming Interface (interface de programação de aplicações).
CI	Continuous Integration (integração contínua).
CF	Cenário funcional do Documento de Visão (por exemplo, CF-03).
CRUD	Create, Read, Update, Delete (criar, consultar, atualizar e excluir).
GQM	Goal-Question-Metric (objetivo–questão–métrica), abordagem de medição adotada no projeto.
Hexad	Modelo de gamificação com seis tipos de usuário: Achiever, Philanthropist, Free Spirit, Player, Socialiser e Disruptor.
JPA	Jakarta Persistence API, especificação de mapeamento objeto-relacional em Java.
JWT	JSON Web Token, token assinado utilizado na autenticação sem estado (stateless).
MoSCoW	Técnica de priorização: Must, Should, Could e Won’t have.
MVC	Model-View-Controller; não caracteriza o Backend REST adotado, que não possui View.
MVP	Minimum Viable Product (produto mínimo viável).

Termo	Significado
ORM	Object-Relational Mapping (mapeamento objeto-relacional).
PR	Neste documento, recorde pessoal (Personal Record) de um exercício. Pull Request é sempre escrito por extenso.
REST	Representational State Transfer, estilo de comunicação entre cliente e servidor sobre HTTP.
RF	Requisito funcional do backlog do Documento de Visão (RF-01 a RF-24).
SOLID	Princípios de projeto orientado a objetos: responsabilidade única, aberto/fechado, substituição de Liskov, segregação de interfaces e inversão de dependência.
SPA	Single Page Application (aplicação de página única).
T1 a T22	Casos de teste planejados no Documento de Visão (seção 6.2).

Fonte: elaborado pelos autores (2026).

1.4 Documentos de referência

Esta versão toma como base o Documento de Visão 1.0.0 e acompanha uma cópia revisada 1.0.1 com as mesmas prioridades, decisões sobre recordes, Hexad e cobertura. Os originais são preservados. As alterações constituem a proposta documental desta revisão e não evidenciam implementação ou homologação pelo Dono do Produto. Mudanças futuras de escopo e cronograma devem atualizar ambos os históricos e matrizes de rastreabilidade.

1.5 Organização do documento

A seção 2 apresenta a representação arquitetural: estilo adotado e justificativa, tecnologias, detalhamento em camadas, metas e restrições, as visões de uso, de organização lógica, estrutural e de implantação, a operação e o ciclo de vida dos dados, a rastreabilidade com o Documento de Visão e os riscos arquiteturais. A seção 3 lista a bibliografia.

2 REPRESENTAÇÃO ARQUITETURAL

2.1 Definições

O sistema adotará o estilo arquitetural Cliente-Servidor (*Client-Server*) e operará em múltiplos níveis (*multi-tier*), conforme descrito por Bass, Clements e Kazman (2021) e por Fowler (2006). Haverá uma separação lógica e física bem definida entre a camada de apresentação consumida pelos usuários (Frontend), a camada responsável pelo processamento de negócio (Backend) e a estrutura de persistência (Banco de Dados). A comunicação entre cliente e servidor ocorrerá exclusivamente por meio de uma API RESTful, com troca de dados em formato JSON, o que padroniza a troca de informações e garante o baixo acoplamento entre as camadas.

2.2 Justificativa da escolha

A escolha pelo estilo Cliente-Servidor com comunicação via API RESTful é justificada pelas necessidades operacionais e de organização do projeto MotivaFit, conforme evidenciado no Documento de Visão:

- **Separação de responsabilidades e desenvolvimento paralelo:** a equipe possui papéis delineados entre desenvolvedores Backend e Frontend. A API REST permite que a interface (React) e os serviços (Java/Spring Boot) sejam construídos em paralelo, desde que o contrato da API seja definido previamente, mitigando riscos de integração.
- **Entregas curtas e incrementais:** o projeto utiliza Sprints semanais e prevê integração contínua. A independência entre Frontend, Backend e Banco de Dados permite testar unidades e integrações isoladamente e implantar cada parte de forma independente.
- **Segurança e isolamento de dados:** o MotivaFit armazena rotinas, cargas, repetições e recordes pessoais. Manter as regras de negócio e o acesso ao banco exclusivamente no servidor permite validar cada requisição, e a autenticação baseada em tokens JWT é compatível com o caráter *stateless* das APIs REST.
- **Evolução e extensibilidade:** funcionalidades Should e Could (painel administrativo, desafios, feed, login com Google) entram como módulos adicionais sem alterar o que já está pronto.

2.2.1 Tecnologias e decisões arquiteturais

A Tabela 4 consolida as tecnologias definidas no Documento de Visão, a camada em que atuam e o custo aceito em cada decisão. Como ferramentas de apoio ao desenvolvimento, o Documento de Visão também prevê Git, GitHub, Figma, VS Code, IntelliJ, Docker, Postman e Maven.

Tabela 4 – Tecnologias e decisões arquiteturais

Tecnologia	Camada	Motivação	Alternativa considerada	Custo aceito
Java / Spring Boot	Backend	Robustez para as regras de negócio.	Node.js / Express	Maior verbosidade em troca de tipagem forte.
React	Frontend	Criação de interface web interativa.	Vue.js	Curva de aprendizado da equipe em troca de ecossistema.

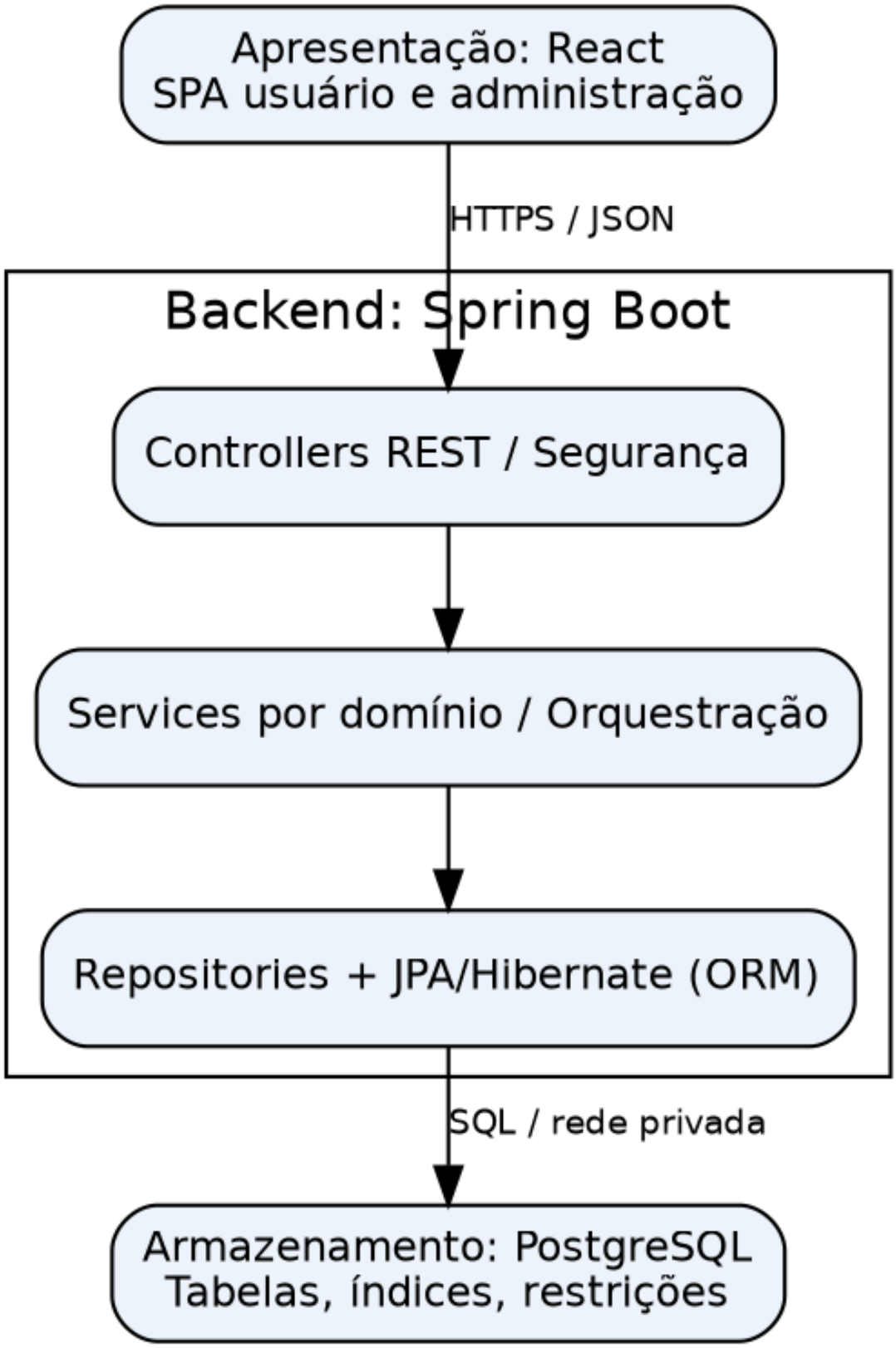
Tecnologia	Camada	Motivação	Alternativa considerada	Custo aceito
PostgreSQL	Persistência	Estruturação relacional de treinos e usuários.	MongoDB	Rigidez do schema em troca de integridade ACID (atomicidade, consistência, isolamento e durabilidade).
Docker / GitHub Actions	Infraestrutura e CI	Padronização dos ambientes e automação de integração contínua.	Implantação manual	Tempo de configuração em troca de confiabilidade.
Git / GitHub	Versionamento	Versionamento colaborativo.	GitLab	Padrão de mercado adotado pela equipe.

Fonte: elaborado pelos autores (2026), com base no Documento de Visão (GRUPO PERLMAN, 2026).

2.3 Detalhamento

O MotivaFit segue Cliente-Servidor em três níveis. React consome JSON por HTTPS; Spring Boot valida autenticação, autorização e regras de domínio. Repositórios e JPA/Hibernate executam no Backend e traduzem operações em SQL. PostgreSQL compõe exclusivamente a camada de armazenamento. A Figura 1 mostra essa localização do ORM.

Figura 1 – Diagrama de arquitetura de software



Fonte: elaborado pelos autores (2026).

2.3.1 Explicação do diagrama em camadas

- **Camada de Apresentação (Frontend):** desenvolvida com a biblioteca React, como aplicação de página única (SPA) com interface para o usuário e interface administrativa. É responsável por gerir as interações do usuário final (cadastro e login, gestão de treinos, registro de exercícios, dashboards de evolução, conquistas e interações sociais). Não contém regras de negócio críticas: todas as informações são obtidas por chamadas HTTP/REST e respostas em JSON.
- Camada de Serviços e Negócio (Backend): Spring Boot, organizado por domínio, com Controllers, Services e Repositories. Inclui autenticação, regras de treinos, cálculo de métricas e recordes, conquistas e desafios. O adaptador JPA/Hibernate pertence ao Backend. Hexad não é um mecanismo de execução.
- Camada de Persistência (Banco de Dados): PostgreSQL mantém tabelas, restrições, índices e transações ACID; é acessado apenas pelo Backend. O ORM não executa no servidor do banco.

2.3.2 Contrato da API REST

A Tabela 5 apresenta a proposta inicial dos recursos expostos pela API, associados aos requisitos do backlog. Os caminhos são indicativos e serão refinados e versionados pela equipe de Backend e Frontend antes da Sprint em que cada requisito for implementado. As requisições e respostas utilizam JSON, os resultados são indicados pelos códigos de status HTTP e as rotas protegidas exigem o token JWT no cabeçalho *Authorization*.

Tabela 5 – Recursos REST previstos

Recurso	Métodos	Requisitos	Acesso
/api/auth/cadastro	POST	RF-01	Público
/api/auth/login	POST	RF-02	Público
/api/auth/google	POST	RF-24	Público (opcional, após o MVP)
/api/treinos	GET, POST	RF-03, RF-04	Usuário autenticado
/api/treinos/{id}	PUT, DELETE	RF-05, RF-06	Usuário autenticado (dono do treino)
/api/treinos/{id}/adicionar	POST	RF-04	Usuário autenticado (treino público)
/api/treinos/{id}/exercicios	GET, POST	RF-07, RF-08	Usuário autenticado (dono do treino)
/api/treinos/{id}/exercicios/{idEx}	PUT, DELETE	RF-09, RF-10	Usuário autenticado (dono do treino)
/api/registros	POST, GET	RF-11	Usuário autenticado
/api/metricas/evolucao	GET	RF-12	Usuário autenticado
/api/catalogo-exercicios/{id}/recordes	GET	RF-20	Autenticado; somente recordes próprios, calculados automaticamente
/api/admin/convites	POST	RF-13	Superusuário
/api/admin/metricas/*	GET	RF-14 a RF-17	Administrador
/api/desafios	GET, POST	RF-18	Usuário autenticado
/api/desafios/{id}	DELETE	RF-19	Usuário autenticado (criador)
/api/publicacoes	GET, POST	RF-21	Usuário autenticado

Recurso	Métodos	Requisitos	Acesso
/api/conquistas	GET	RF-22	Usuário autenticado
/api/grupos	GET, POST	RF-23	Autenticado: criar grupo e listar grupos próprios
/api/auth/refresh; /api/auth/logout	POST	RF-02 / transversal	Cookie seguro + proteção CSRF; rotação/revogação
/api/conta; /api/conta/exportacao	DELETE; GET	Privacidade / transversal	Titular; DELETE com autenticação recente
/api/catalogo-exercicios; /api/catalogo-exercicios/{id}	GET	RF-07	Todos os perfis autenticados; somente ativos
/api/admin/catalogo-exercicios; /api/admin/catalogo-exercicios/{id}	POST; PUT, DELETE	Suporte RF-07	Administrador ou superusuário; DELETE desativa
/api/admin/convites/aceite	POST	RF-13	Token de convite válido + e-mail convidado verificado
/api/amizades; /api/amizades/{id}	GET, POST; PATCH, DELETE	Suporte RF-18	Solicitante/destinatário; PATCH aceita ou bloqueia
/api/desafios/{id}/participantes; /api/desafios/{id}/participantes/me	GET, POST; PATCH, DELETE	RF-18, RF-19	Criador convida amigos; destinatário aceita/recusa ou sai
/api/grupos/{id}	GET, PUT, DELETE	RF-23	Membro consulta; proprietário altera/exclui
/api/grupos/{id}/membros; /api/grupos/{id}/membros/{usuarioId}	GET; DELETE	RF-23	Membro lista; próprio membro sai ou proprietário remove
/api/grupos/{id}/convites; /api/grupos/convites/aceite	POST; POST	RF-23	Proprietário convida; destinatário aceita token válido
/api/grupos/{id}/feed	GET	RF-23	Somente membro
/api/publicacoes/{id}	GET, DELETE	RF-21, RF-23	Leitura conforme visibilidade; exclusão pelo autor
/api/publicacoes/{id}/curtidas/me	PUT, DELETE	RF-21, RF-23	Autenticado com acesso à publicação; operação idempotente
/api/publicacoes/{id}/comentarios; /api/comentarios/{id}	GET, POST; PUT, DELETE	RF-21, RF-23	Acesso à publicação; alteração/exclusão pelo autor
/api/registros/{id}	GET, PUT, DELETE	RF-11 / correção técnica	Dono; correções recalculam métricas e projeções
/api/admin/conquistas; /api/admin/conquistas/{id}	POST; PUT	Suporte RF-22	Administrador; critérios versionados

Fonte: elaborado pelos autores (2026). Proposta inicial a ser refinada nas Sprints.

2.3.3 Mecanismos de segurança

- Autenticação: login com e-mail e senha emite JWT de acesso com validade de 15 minutos, mantido em memória pelo cliente. Refresh token opaco com validade de 7 dias fica em cookie HttpOnly, Secure e SameSite=Lax, com hash persistido no Backend; é rotacionado a cada renovação. Reuso revoga a família de tokens. Logout e exclusão de conta revogam as renovações; o filtro valida também conta ativa e versão de credencial para impedir acesso com JWT remanescente. Esses prazos são decisões propostas, a validar nos testes.
- **Armazenamento de senhas:** apenas o *hash* da senha é persistido, gerado por algoritmo adaptativo (por exemplo, BCrypt); a senha em texto puro nunca é registrada.
- **Autorização:** controle por perfil (usuário, administrador e superusuário) e por posse do recurso. Um usuário só acessa os próprios registros e os treinos explicitamente públicos de outros usuários (T14); as rotas administrativas são negadas a usuários comuns (T11).
- **Validação e mensagens de erro:** os dados de entrada são validados no Backend antes da persistência, e as mensagens de erro não revelam informações sensíveis (T3 e T5).

- **Transporte:** toda comunicação entre navegador e servidores ocorre por HTTPS; os segredos (chave de assinatura do JWT, credenciais do banco) são fornecidos por variáveis de ambiente e não versionados no repositório.

CORS: permitir somente as origens exatas cadastradas para desenvolvimento e futura hospedagem; nunca usar origem curinga com credenciais. Métodos e cabeçalhos são explicitamente permitidos. Refresh e logout validam Origin e token anti-CSRF, pois usam cookie. CORS não substitui autenticação e autorização.
Inicialização administrativa: três perfis (USUARIO, ADMINISTRADOR, SUPERUSUARIO). O superusuário herda as permissões administrativas. O primeiro é criado por comando de bootstrap executado pelo operador, com credenciais fornecidas por segredo e senha temporária que deve ser trocada. O comando é idempotente, não é uma rota pública, não possui senha padrão e registra auditoria sem expor segredos. O cadastro público sempre cria USUARIO; apenas o superusuário concede ADMINISTRADOR por convite.

2.4 Metas e restrições arquiteturais

As metas e restrições arquiteturais foram definidas considerando o contexto do sistema, os requisitos funcionais do Documento de Visão, as tecnologias adotadas pela equipe e o perfil dos usuários. Seu objetivo é garantir que a arquitetura seja segura, modular, testável e adequada à evolução incremental do produto ao longo das Sprints.

2.4.1 Metas arquiteturais

2.4.1.1 Garantir modularidade e baixo acoplamento

A arquitetura deve manter separação clara entre interface, regras de negócio e persistência, e organizar o Backend em módulos por domínio, aplicando os princípios SOLID e Clean Code. Cada módulo pode ser desenvolvido, testado e evoluído de forma independente, o que facilita a prototipagem rápida e minimiza os impactos de mudanças no layout do aplicativo.

2.4.1.2 Garantir qualidade contínua

Manter testes automatizados relevantes. A meta M3 permanece cobertura global ponderada de linhas ≥ 70%, medida também separadamente no Backend e Frontend. Sua viabilidade desde o primeiro Pull Request não está confirmada: esta revisão dispõe de documentação, sem resultados do pipeline. Até o piloto na Sprint 3 a cobertura é informativa; testes funcionais continuam bloqueantes. Antes do encerramento do MVP, a equipe deve executar um Pull Request representativo com builds, testes e relatórios, registrar cobertura, duração e esforço, e ativar o gate de 70% após demonstrar viabilidade. Sem evidência, a decisão fica pendente e deve ser revista nos dois documentos, sem alegar cumprimento. M2 e M4 medem defeitos e dívida técnica, não provam modularidade.

2.4.1.3 Garantir segurança e controle de acesso

Como o MotivaFit armazena rotinas, evolução física (cargas, repetições) e recordes pessoais, é vital proteger a privacidade desses registros. A camada de serviços deve validar estritamente as requisições, garantindo que um atleta tenha acesso exclusivo às suas próprias métricas e apenas às interações permitidas nos grupos e espaços públicos do aplicativo.

2.4.1.4 Garantir usabilidade e fluidez

O registro diário deve ser rápido e claro. Metas propostas de desempenho: p95 ≤ 500 ms para leitura e ≤ 800 ms para gravação de registros, erros de servidor < 1%, em ensaio de 10 minutos com 20 usuários simultâneos, banco com 100 mil registros e servidor de referência com 2 vCPU e 4 GiB de RAM. Medir após aquecimento de 2 minutos, reportando infraestrutura, massa e latência; esses são critérios de aceitação, não resultados medidos. Usabilidade: realizar duas rodadas (MVP e após recursos sociais), com pelo menos cinco participantes representativos por rodada, em celular e computador; tarefas de cadastrar, montar treino, registrar e consultar histórico, com ≥ 90% de conclusão sem ajuda e mediana ≤ 60 segundos para registrar execução em conta preparada. T20 cobre a primeira rodada; repetir após mudanças de fluxo e documentar problemas e correções.

2.4.1.5 Garantir evolução e extensibilidade

A arquitetura deve permitir que novas funcionalidades sejam acrescentadas sem alterar o que já está pronto. O planejamento do Documento de Visão depende disso: o MVP fecha na Sprint 4 e os recursos Should e Could chegam depois, conforme a capacidade da equipe. Cada recurso novo entra como módulo ou componente adicional (Sprints semanais conforme o Guia do Scrum, de Schwaber e Sutherland, 2020); o login com Google, por exemplo, soma-se ao acesso por e-mail e senha sem substituí-lo.

A Tabela 6 distingue métricas arquiteturais propostas nesta revisão das métricas de processo M1 a M4 do Documento de Visão. Vazão mede entregas e dívida técnica mede problemas estáticos; nenhuma delas, isoladamente, verifica extensibilidade ou modularidade.

Tabela 6 – Metas arquiteturais e verificação

Meta	Como é verificada	Referência (Visão)
2.4.1.1 Modularidade	Testes arquiteturais por Pull Request: zero ciclos entre domínios e zero acesso a repositórios internos de outro módulo; verificar dependências da Figura 4.	MA1 (nova); M4 complementa
2.4.1.2 Qualidade	Builds/testes bloqueantes; M3 global ≥ 70% após piloto de viabilidade; registrar cobertura por camada. M2 ≤ 0,2 bugs/issue por Sprint.	M2, M3; piloto Sprint 3
2.4.1.3 Segurança	Testar isolamento entre contas e grupos, os três perfis, bootstrap, rotação/reuso de refresh, CSRF e exclusão de conta.	T3, T11, T14, T21, T22 + casos técnicos
2.4.1.4 Usabilidade/desempenho	Duas rodadas com ≥ 5 participantes; conclusão ≥ 90%; registro mediano ≤ 60 s. Carga de 20 usuários: p95 leitura ≤ 500 ms e gravação ≤ 800 ms; erros < 1%.	T20 ampliado; MA2/MA3 novas
2.4.1.5 Extensibilidade	Em cada novo recurso, registrar módulos alterados e mudanças de contrato; zero mudanças incompatíveis não versionadas. Exercício com nova regra de conquista deve mudar somente Gamificação e sua definição, preservando Core e contrato de Registro.	MA4 (nova); M1 mede processo

Fonte: elaborado pelos autores (2026), com base no Documento de Visão (GRUPO PERLMAN, 2026).

2.4.2 Restrições arquiteturais

As restrições arquiteturais definem limitações, padrões e diretrizes técnicas que devem ser seguidos durante o desenvolvimento.

2.4.2.1 Restrição tecnológica

O projeto deve utilizar as tecnologias estipuladas no Documento de Visão: Java com Spring Boot no Backend, React no Frontend e PostgreSQL na persistência, com Docker e GitHub Actions para containerização e integração contínua e Git/GitHub para versionamento.

2.4.2.2 Restrição de estilo arquitetural

O sistema deve seguir o modelo Cliente-Servidor em múltiplas camadas. A comunicação entre Frontend e Backend ocorre por API RESTful e a autenticação utiliza tokens JWT para o controle de acesso às funcionalidades protegidas. O banco de dados é acessado exclusivamente pelo Backend.

2.4.2.3 Restrição de padronização de código

- Backend: seguir os princípios SOLID e as diretrizes de Clean Code, com as convenções de nomenclatura da linguagem Java (classes em PascalCase; métodos e variáveis em camelCase) e organização em pacotes por domínio, evitando débito técnico a longo prazo.
- Frontend: componentes React com responsabilidade única, arquivos organizados de forma modular e reutilizável e comunicação com a API concentrada em serviços de requisição.
- A conformidade será verificada por revisões de código (*Code Review* e *Pair Programming*) e por análise estática automatizada.

2.4.2.4 Restrição de versionamento e integração

Cada funcionalidade é desenvolvida em branch própria e integrada por Pull Request. Builds, testes e análise estática crítica são bloqueantes. A política de cobertura e o piloto de viabilidade seguem a seção 2.4.1.2, sem impor um bloqueio sem baseline comprovada. A análise estática roda a cada Pull Request e em execução diária independente.

2.4.2.5 Restrição operacional e acadêmica

- O desenvolvimento ocorre em Sprints semanais, sem Daily Scrum, pois a disponibilidade dos integrantes impossibilita encontros diários; o acompanhamento é feito por reuniões semanais e pelos canais de comunicação do grupo.
- As funcionalidades são implementadas conforme a priorização MoSCoW do backlog.
- O mesmo ambiente base (Linux e containers Docker) deve ser usado por todos os integrantes, garantindo reprodutibilidade.

2.5 Visões

Esta seção apresenta as visões de uso, de organização lógica e estrutural do sistema. Por meio dos diagramas e das descrições é possível compreender como os usuários interagem com o sistema e como os requisitos do projeto influenciaram as decisões arquiteturais. O contexto do MotivaFit abrange praticantes de exercícios, iniciantes e atletas, e os principais grupos de casos de uso que influenciaram a arquitetura são:

- **Gestão de treinos:** criar, alterar, listar e excluir treinos e seus exercícios (RF-03 a RF-10).

- **Registro diário:** registrar os exercícios realizados com carga, repetições, séries e tempo (RF-11).
- **Análise:** consultar indicadores de evolução e recordes pessoais (RF-12 e RF-20).
- **Gamificação e interação social:** conquistas por assiduidade, desafios, compartilhamento, grupos e feed (RF-18 a RF-23).
- **Administrativo:** convite de administradores e métricas da plataforma sobre retenção e rotinas mais usadas (RF-13 a RF-17).

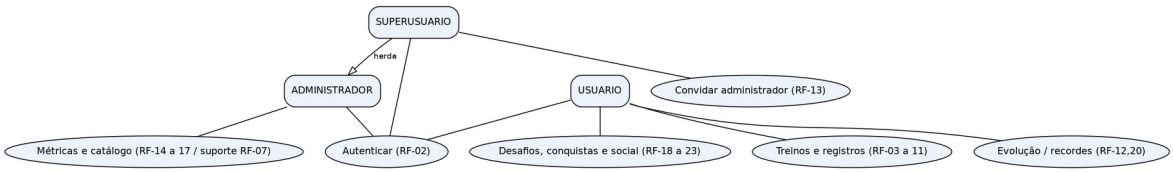
2.5.1 Visão de uso

O MotivaFit possui três perfis: USUARIO utiliza treinos e métricas pessoais; ADMINISTRADOR consulta métricas agregadas e mantém o catálogo; SUPERUSUARIO herda essas atribuições e convida administradores. Todos autenticam pelo mesmo módulo. O bootstrap do primeiro superusuário está definido na seção 2.3.3.

2.5.1.1 Diagrama de casos de uso

A Figura 2 associa os três perfis à autenticação e delimita permissões. O superusuário herda as permissões do administrador; somente ele cria convites administrativos. Indicadores administrativos são agregados e não liberam automaticamente acesso ao histórico individual dos usuários.

Figura 2 – Diagrama de casos de uso do sistema MotivaFit



Fonte: elaborado pelos autores (2026).

Tabela 7 – Casos de uso, atores e requisitos

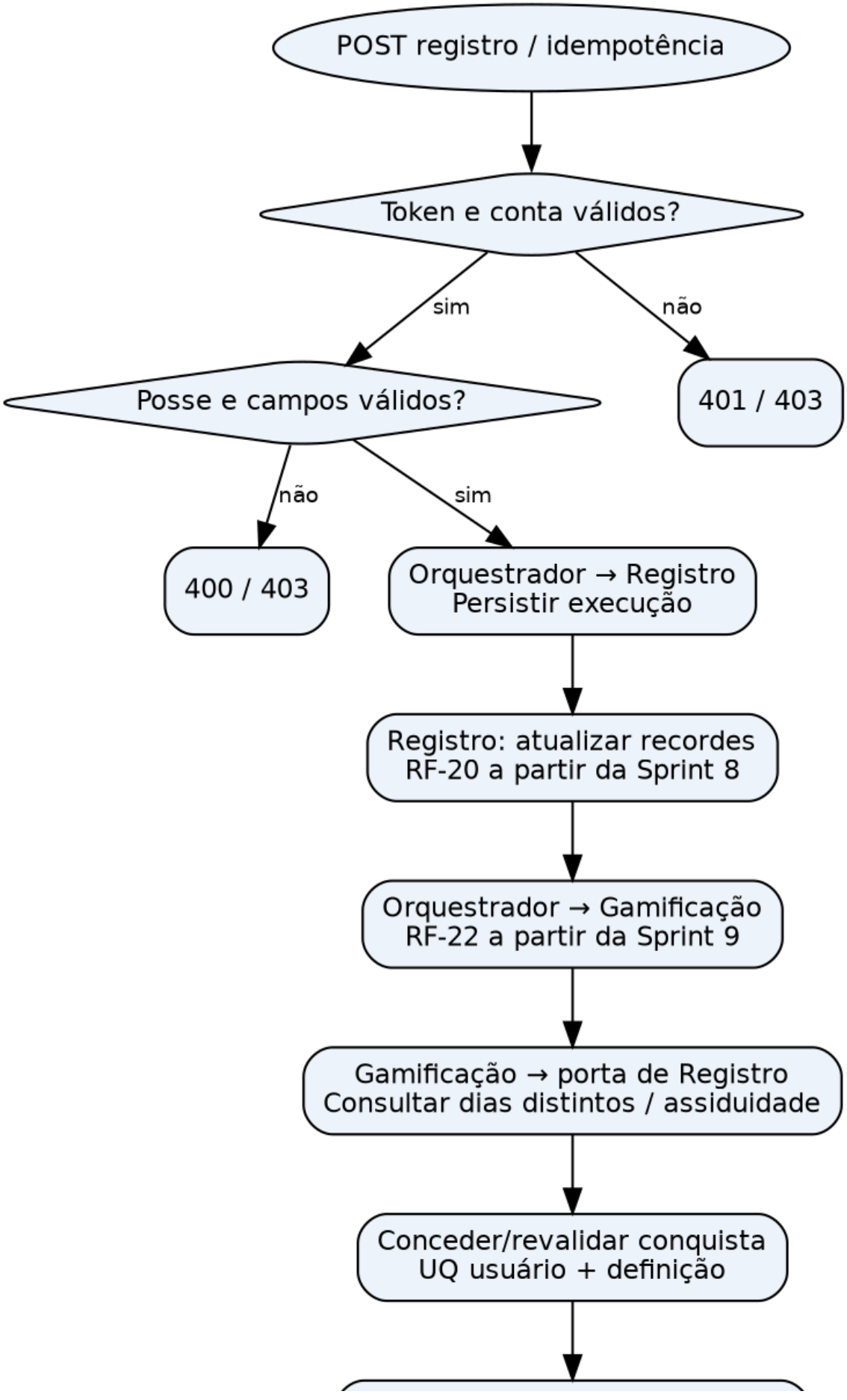
Caso de uso	Ator	Requisitos	Sprint	Prioridade
Cadastrar conta	Usuário	RF-01	3	Must
Autenticar com e-mail e senha	Usuário, Administrador, Superusuário	RF-02	3	Must
Gerenciar treinos	Usuário	RF-03 a RF-06	3	Must
Gerenciar exercícios do treino	Usuário	RF-07 a RF-10	4	Must
Registrar exercício realizado	Usuário	RF-11	4	Must
Acompanhar métricas e recordes pessoais	Usuário	RF-12, RF-20	5 e 8	Should
Convidar administrador por e-mail	Superusuário	RF-13	5	Could
Consultar painel administrativo	Administrador	RF-14 a RF-17	5 a 7	Should
Criar e remover desafios	Usuário	RF-18, RF-19	7	Should
Compartilhar resultados	Usuário	RF-21	8	Should
Visualizar conquistas	Usuário	RF-22	9	Should
Participar de grupos e feed	Usuário	RF-23	9	Should
Autenticar com Google (opcional)	Usuário	RF-24	9 e 10	Could

Fonte: elaborado pelos autores (2026), com base no Documento de Visão (GRUPO PERLMAN, 2026).

2.5.1.2 Diagrama de atividades

A Figura 3 mostra a orquestração do registro: autenticar e validar posse e campos, persistir a execução e atualizar projeções que já tenham sido entregues. Na Sprint 4 somente o registro faz parte do MVP; recordes e conquistas entram nas Sprints 8 e 9. O orquestrador de aplicação chama as interfaces de Registro e Gamificação; o domínio Registro não depende do domínio Gamificação.

Figura 3 – Diagrama de atividades do registro de exercício e gamificação


POST registro / idempotência
Token e conta válidos?
sim
não
Posse e campos válidos?
401 / 403
não
sim
400 / 403
Orquestrador → Registro
Persistir execução
Registro: atualizar recordes
RF-20 a partir da Sprint 8
Orquestrador → Gamificação
RF-22 a partir da Sprint 9
Gamificação → porta de Registro
Consultar dias distintos / assiduidade
Conceder/revalidar conquista
UQ usuário + definição

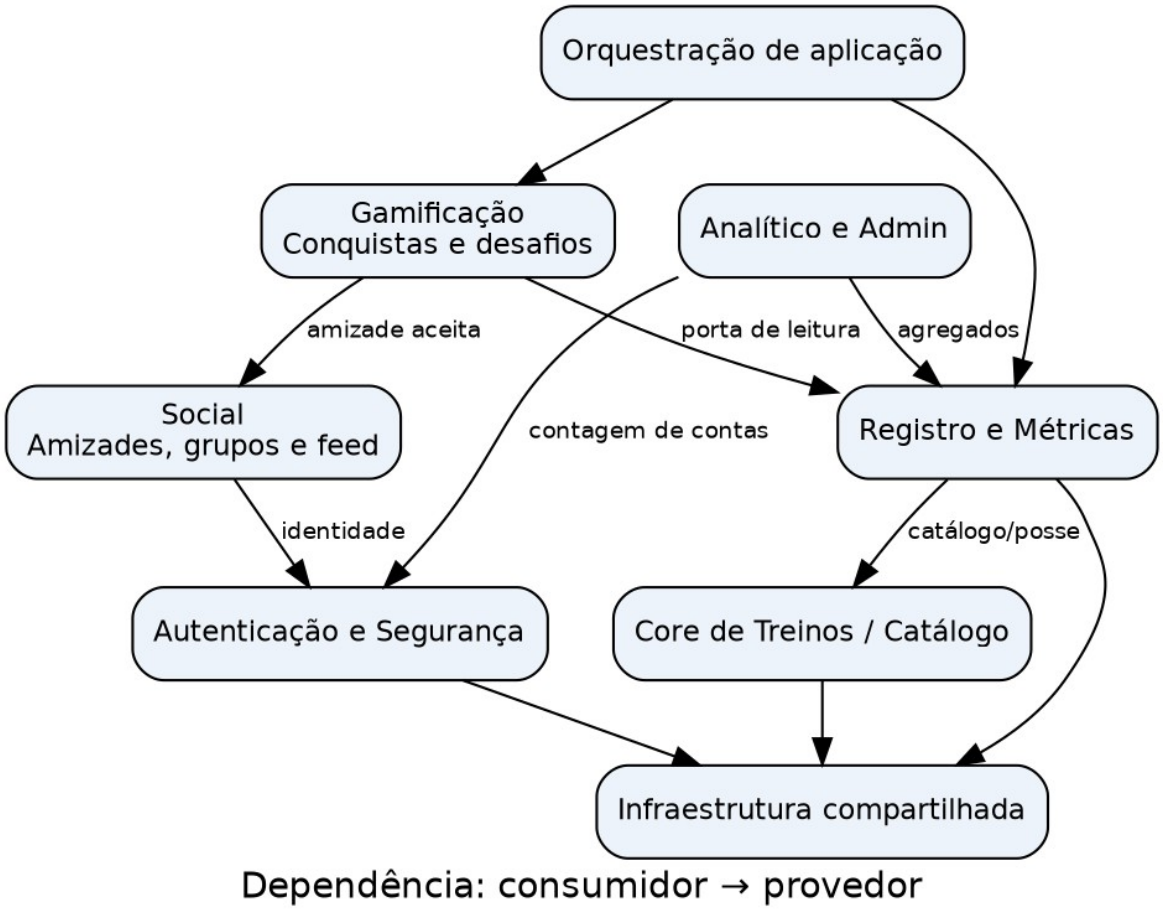
Fonte: elaborado pelos autores (2026).

Na mesma transação local, o orquestrador persiste o registro e, quando disponíveis, chama atualização de recordes e avaliação de conquistas. Gamificação consulta a interface de leitura de Registro e Métricas para apurar assiduidade; Registro não chama Gamificação. A API responde com os dados dos recursos já entregues. Falhas provocam rollback; restrições únicas e chave de idempotência impedem duplicação na repetição de requisições. As Figuras 3, 4 e 7 e a Tabela 14 usam o mesmo sentido de dependência (consumidor → provedor).

2.5.2 Visão de organização lógica

A organização lógica do MotivaFit divide o sistema em módulos que agrupam funcionalidades relacionadas ao domínio, seguindo o princípio de alta coesão e baixo acoplamento. Essa divisão reduz o acoplamento, facilita a manutenção, permite o desenvolvimento paralelo entre os integrantes e favorece a evolução incremental ao longo das Sprints. O Frontend concentra os módulos de interface, navegação e comunicação com a API; o Backend concentra as regras de negócio, a autenticação e a persistência. O acesso ao banco ocorre exclusivamente pelo Backend, por meio do ORM, evitando o acesso direto da interface aos dados.

Figura 4 – Visão de organização lógica em módulos



Fonte: elaborado pelos autores (2026).

Tabela 8 – Módulos lógicos do Backend e requisitos atendidos

Módulo	Responsabilidades	Requisitos
1. Autenticação e Segurança	Cadastro de contas, autenticação por e-mail e senha, emissão e validação de tokens JWT, perfis de acesso, convite de administradores e, futuramente, login opcional via Google (OAuth).	RF-01, RF-02, RF-13, RF-24
2. Core de Treinos	Treinos do usuário, sua composição por exercícios (selecionados no catálogo predefinido), metas por exercício e adição de treinos já montados.	RF-03 a RF-10
3. Registro e Métricas	Registro diário de execução (carga, repetições, séries e tempo), indicadores de desempenho e evolução e recordes pessoais.	RF-11, RF-12, RF-20
4. Gamificação	Conquistas por assiduidade e desafios; consulta a interface de leitura de Registro. Hexad é referência de projeto.	RF-18, RF-19, RF-22
5. Analítico e Admin	Compilações de dados para o painel: usuários ativos, frequência média, ranking por tempo de treino e exercícios mais realizados.	RF-14 a RF-17
6. Infraestrutura compartilhada	Configurações, validações, tratamento de erros, filtros de segurança e padrão de resposta da API, reutilizados pelos demais módulos.	Transversal
7. Social	Amizades, publicações, curtidas, comentários, grupos, membros e convites.	RF-21, RF-23

Fonte: elaborado pelos autores (2026), com base no Documento de Visão (GRUPO PERLMAN, 2026).

Hexad foi usado somente como referência de projeto, distribuída entre vários módulos. Os seis tipos são discutidos abaixo; não se promete atender integralmente todos eles. Marczewski (2015) e Tondello et al. (2016) relacionam Philanthropist a propósito e altruísmo e Disruptor a mudança do sistema; mecânicas sociais ou desafios não demonstram essas motivações por si só.

- Achiever: maestria e progressão, apoiadas por métricas e recordes (Registro e Métricas, RF-12 e RF-20).
- Free Spirit: autonomia para compor rotinas e metas (Core de Treinos, RF-03, RF-05, RF-07 e RF-09).
- Player: recompensas visuais de assiduidade (Gamificação, RF-22).
- Philanthropist: o compartilhamento de RF-21 pode apoiar outros usuários se for usado com intenção de ajuda, uma hipótese de projeto inspirada em propósito e altruísmo na literatura. Compartilhar resultados sozinho não comprova esse tipo; não existe mecânica específica de mentoria ou doação no escopo.
- Socialiser: relações e interação em grupos e feed (Social, RF-21 e RF-23).
- Disruptor: a literatura associa esse tipo a mudança e influência sobre o sistema. Criar desafios (RF-18) não permite alterar regras da plataforma, portanto o mapeamento anterior é removido. O escopo não atende especificamente esse tipo; desafios são mecânicas de progressão e interação social. Não se adiciona questionário ou requisito para preencher artificialmente essa lacuna.

2.5.3 Visão estrutural

O Backend usa arquitetura em camadas organizada por domínio (Controllers → Services → Repositories), e não MVC completo: a API não possui View. JSON é um contrato de transporte, enquanto a apresentação é responsabilidade do React. Cada pacote de domínio contém seus controladores, serviços, portas e adaptadores de persistência; chamadas entre domínios usam interfaces públicas, nunca repositórios ou entidades internas de outro módulo.

- **Controllers:** recebem as requisições HTTP e expõem os endpoints no formato JSON.

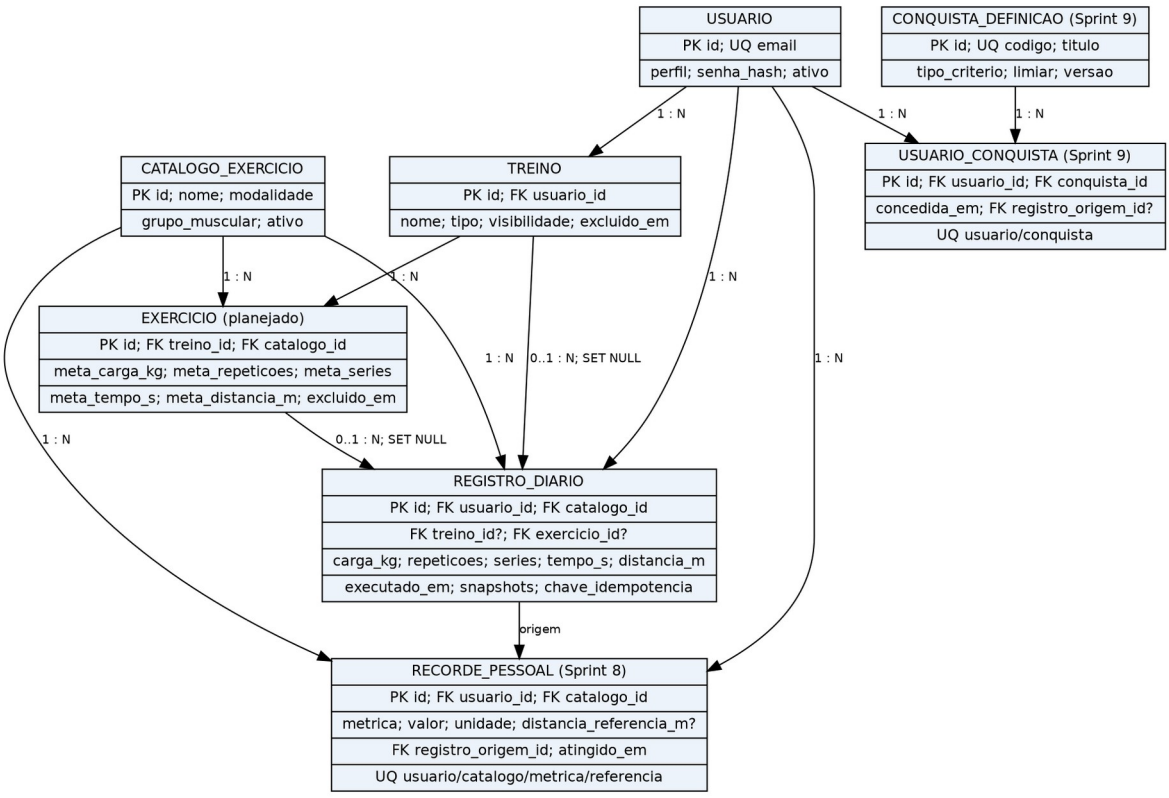
- **Services:** concentram toda a lógica de negócio, inclusive as regras de gamificação.
- **Repositories:** realizam as consultas e as abstrações de acesso ao banco PostgreSQL.
- **Entidades/Modelos:** representam os dados do domínio (Usuário, Treino, Exercício, Registro Diário, Conquista e demais) mapeados pelo ORM.

Essa separação favorece a testabilidade: as regras presentes nos Services podem ser testadas de forma isolada, e a integração entre Controllers, Services, Repositories e banco de dados pode ser validada por testes de integração. O Frontend é estruturado com componentização funcional em React, orientada a fluxos: páginas, componentes de interface e serviços de requisição.

2.5.3.1 Modelo de dados relacional

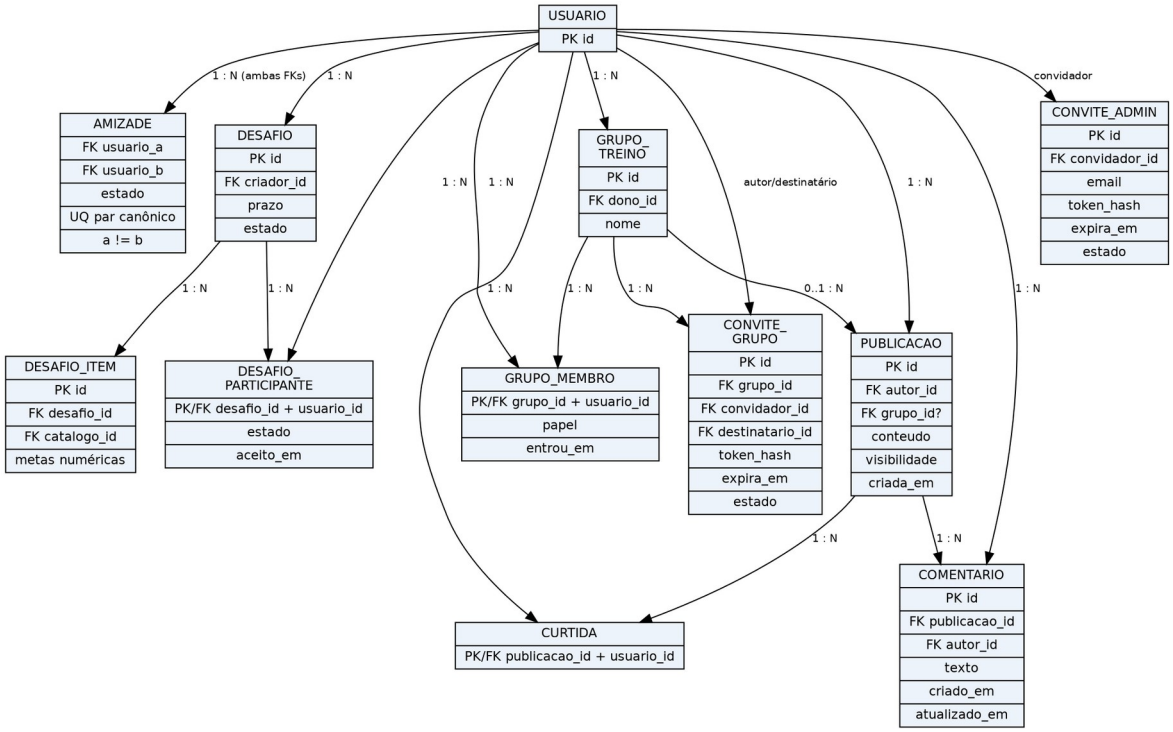
As Figuras 5 e 6 são modelos de dados relacionais, não diagramas de classes UML. A Figura 5 distingue catálogo, exercício planejado, execução histórica, recorde numérico e concessão de conquista. A Figura 6 define desafios e participantes, amizade, grupos, membros, convites e conteúdo social. PK identifica chave primária, FK chave estrangeira e UQ unicidade; o texto a seguir complementa cardinalidades, unidades e regras de integridade. As entidades posteriores ao MVP são identificadas pela Sprint na Tabela 9.

Figura 5 – Modelo de dados: MVP, recordes (Sprint 8) e conquistas (Sprint 9)



Fonte: elaborado pelos autores (2026).

Figura 6 – Modelo de dados relacional: entidades previstas para as Sprints 5 a 9



Fonte: elaborado pelos autores (2026).

Tabela 9 – Entidades do domínio

Entidade	Descrição	Requisitos	Sprint
USUARIO	Conta de acesso. Possui e-mail único, hash da senha e perfil (usuário, administrador ou superusuário).	RF-01, RF-02	3
TREINO	Rotina do usuário, com nome, tipo (musculação ou aeróbico) e visibilidade (privado ou público), esta última para permitir a adição de treinos já montados sem expor treinos privados.	RF-03 a RF-06	3
CATALOGO_EXERCICIO	Tabela de exercícios predefinidos, com nome e grupo muscular. Não é editável pelo usuário.	RF-07	4
EXERCICIO	Vínculo catálogo/treino; metas numéricas de carga, repetições, séries, tempo e distância; excluido_em. Não guarda recorde.	RF-07 a RF-10	4
REGISTRO_DIARIO	Execução do usuário ligada ao catálogo, com carga_kg, repetições, séries, tempo_s, distancia_m, data e snapshots; FKs opcionais de rotina não apagam histórico.	RF-11, RF-12	4 e 5
CONQUISTA_DEFINICAO e USUARIO_CONQUISTA	CONQUISTA_DEFINICAO descreve critério; USUARIO_CONQUISTA registra concessão única por usuário/definição.	RF-22	9
DESAFIO	Desafio com criador, prazo, itens numéricos e participantes convidados/aceitos; amizade aceita valida convite.	RF-18, RF-19	7
PUBLICACAO, CURTIDA e COMENTARIO	PUBLICACAO, CURTIDA e COMENTARIO separados; visibilidade pública ou restrita a membros de grupo.	RF-21, RF-23	8 e 9
GRUPO_TREINO	Grupo com GRUPO_MEMBRO e CONVITE_GRUPO (destinatário, hash do token, validade e estado).	RF-23	9
CONVITE_ADMIN	Convite de cadastro administrativo com destinatário, hash de token, expiração, estado e convidador superusuário.	RF-13	5
RECORDE_PESSOAL	Projeção numérica automática por usuário, catálogo,	RF-20	8

Entidade	Descrição	Requisitos	Sprint
	métrica e distância de referência quando aplicável.		
AMIZADE e DESAFIO_PARTICIPANTE	Relações de amizade e adesão explícita aos desafios; chaves únicas impedem duplicação.	RF-18	7
REFRESH_TOKEN	Hash, família, usuário, validade e revogação para renovação rotativa.	Transversal RF-02	3

Fonte: elaborado pelos autores (2026), com base no Documento de Visão (GRUPO PERLMAN, 2026).

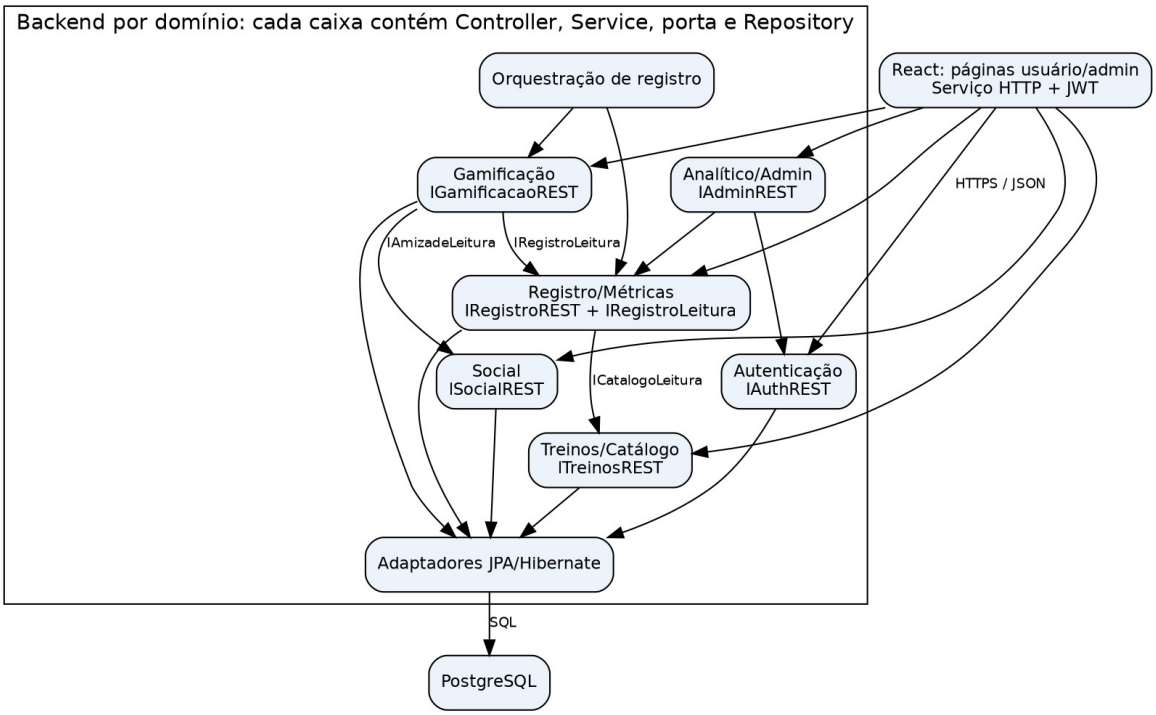
Integridade histórica: excluir treino ou exercício planejado arquiva o item com excluido_em e remove-o das listagens ativas. REGISTRO_DIARIO não sofre exclusão em cascata: mantém usuario_id e catalogo_id obrigatórios, treino_id e exercicio_id opcionais, além de nome do treino e exercício no momento da execução. Se houver expurgo físico da rotina, essas duas FKs usam ON DELETE SET NULL. O catálogo é desativado, nunca excluído enquanto referenciado. Copiar treino público cria nova rotina sem copiar registros ou recordes. A exclusão de conta é um fluxo separado de eliminação de dados pessoais, descrito na seção 2.7.

Campos e recordes: metas e registros usam carga_kg NUMERIC(8,2), repeticoes e series INTEGER, tempo_s INTEGER e distancia_m NUMERIC(10,2), com validação por modalidade; valores não aplicáveis ficam nulos. Tempo, distância, séries e repetições são positivos; carga pode ser zero. Pace em segundos/km é derivado de tempo_s × 1000 / distancia_m, somente com distância > 0. Metas de pace usam meta_tempo_s e meta_distancia_m; não armazenam texto livre. RECORDE_PESSOAL tem usuario_id, catalogo_id, tipo_metrica, valor NUMERIC(12,3), unidade, atingido_em e registro_origem_id, com UQ(usuario_id, catalogo_id, tipo_metrica, distancia_referencia_m) e tratamento de nulos como iguais (UNIQUE NULLS NOT DISTINCT), evitando múltiplos recordes sem referência de distância. Máximos de carga, repetições, distância e duração são independentes; menor pace é melhor. Comparações de pace e duração exigem distância de referência igual (incluída na chave de comparação), evitando comparar corridas diferentes. Não se estima 1RM. Recordes são calculados automaticamente sobre execuções válidas e recalculados ao corrigir ou eliminar um registro; PUT manual de recorde é removido. A mesma execução do catálogo em dois treinos atualiza o mesmo recorde, sem duplicação.
Conquistas: CONQUISTA_DEFINICAO contém código único, título, tipo_criterio, limiar e versão; USUARIO_CONQUISTA contém usuario_id, conquista_id, concedida_em e registro_origem_id opcional, com UQ(usuario_id, conquista_id). Assiduidade usa dias distintos de execução no fuso America/Sao_Paulo; múltiplos registros no mesmo dia contam uma vez. Correções que invalidem o critério revogam a concessão derivada, mantendo auditoria. Catálogo e definições são mantidos por administradores em rotas auditadas, com carga inicial versionada; usuários apenas consultam e selecionam.
Interação social: AMIZADE usa par canônico de usuários distintos e estado pendente/aceita/bloqueada, com unicidade do par. DESAFIO tem criador, prazo e estado; DESAFIO_ITEM liga catálogo e metas numéricas; DESAFIO_PARTICIPANTE tem chave (desafio_id, usuario_id), estado convidado/aceito/recusado e data de aceite. Só amigos aceitos recebem convite; apenas aceitos contribuem para o progresso. Remover desafio o cancela sem remover execuções históricas. GRUPO_MEMBRO usa chave (grupo_id, usuario_id) e papel; CONVITE_GRUPO contém grupo, convidador, destinatário, hash de token, expiração e estado. Aceite valida destinatário e validade e cria associação em transação idempotente. CONVITE_ADMIN segue o mesmo padrão, restrito ao e-mail convidado e sem conceder superusuário. PUBLICACAO tem autor, grupo opcional e visibilidade; CURTIDA é única por publicação/usuário e COMENTARIO tem autor, texto e datas. Grupos privados e seus conteúdos são acessíveis apenas a membros.

2.5.3.2 Diagrama de componentes

A Figura 7 mostra explicitamente as interfaces REST de autenticação, treinos/catálogo, registros/métricas, gamificação, social e administração. O serviço HTTP do React consome essas interfaces. Cada domínio do Backend contém Controller, Service, porta pública e Repository JPA. A seta de dependência aponta do consumidor ao provedor; Analítico e Admin consulta portas de leitura de Registro e Autenticação, e não recebe chamadas desses domínios. ORM é adaptador do Backend.

Figura 7 – Diagrama de componentes



Fonte: elaborado pelos autores (2026).

- **Frontend (React):** interface do usuário, interface administrativa e serviços de requisição; comunica-se com o Backend exclusivamente por API REST em JSON.
- Backend (Spring Boot): domínios Autenticação, Core de Treinos, Registro e Métricas, Gamificação, Social e Analítico e Admin, cada um com seus controladores, serviços, interfaces e repositórios. Orquestração de registro coordena operações entre domínios sem criar dependência inversa.
- **Banco de Dados (PostgreSQL):** armazenamento persistente de todos os dados, acessado apenas pelo Backend.

2.6 Visão de Implantação

Docker padroniza dependências e torna ambientes de desenvolvimento e testes reproduzíveis em Linux. Isso reduz divergências de configuração; Pair Programming depende da organização das duplas e das ferramentas de colaboração, não do contêiner. Esta arquitetura define integração contínua e geração de artefatos; não promete entrega contínua ou deploy automatizado.

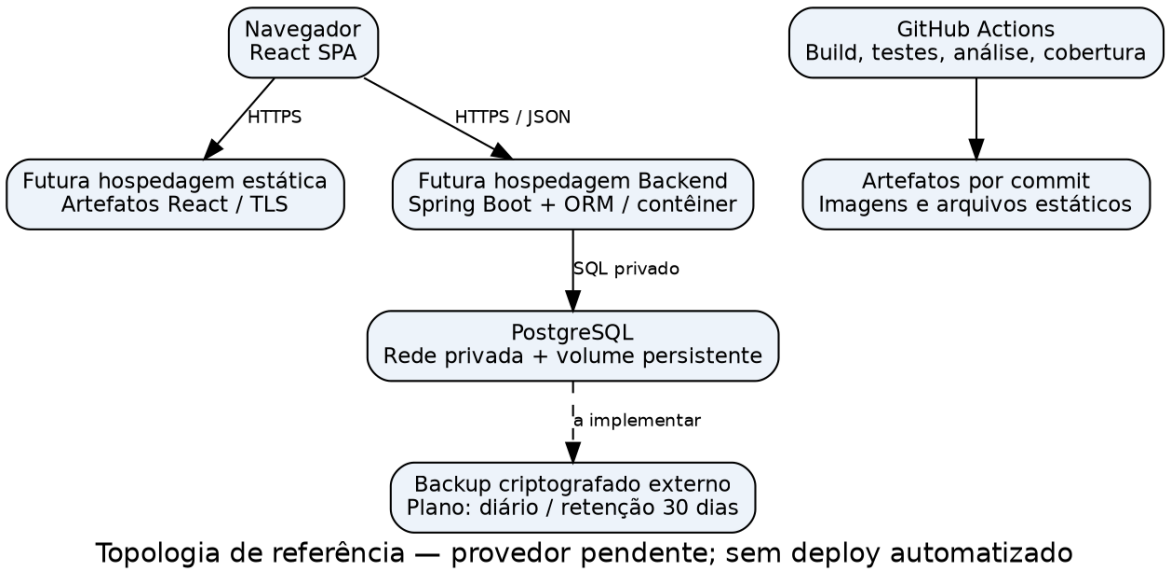
Tabela 10 – Elementos de implantação

Elemento	Descrição
Dispositivo do usuário (navegador web)	Ponto de acesso ao sistema, sem instalação de software. A interface deve se adaptar a telas de celular e de computador.
Hospedagem do Frontend	Serve os artefatos estáticos da aplicação React, gerados pelo build automatizado na esteira de CI.
Backend (contêiner Docker)	Empacota o projeto Spring Boot e o ambiente Java necessário; expõe a API REST por HTTPS.
Banco de Dados (contêiner Docker)	Contêiner com PostgreSQL instanciado com as variáveis de ambiente necessárias, acessível apenas pelo Backend em rede privada.
GitHub e GitHub Actions	Repositório de código e esteira de CI: compilação, testes, cobertura, análise estática e geração das imagens Docker.

Fonte: elaborado pelos autores (2026).

O provedor de hospedagem ainda não foi escolhido. A Figura 8 descreve uma topologia de referência, não uma implantação existente. Ambientes previstos: desenvolvimento local, CI efêmero com dados fictícios e futura homologação separada de produção. Antes da primeira publicação, a equipe deve escolher provedor e responsável operacional, configurar TLS e segredos, definir deploy e rollback e registrar o procedimento. Não há etapa de deploy automatizado nesta revisão.

Figura 8 – Diagrama de implantação



Fonte: elaborado pelos autores (2026).

Na topologia proposta, o navegador carrega React por HTTPS e acessa a API Spring Boot. PostgreSQL fica em rede privada, com volume persistente; contêiner não substitui persistência ou backup. A CI compila e testa, produzindo artefatos rastreáveis pelo commit; sua aprovação não publica automaticamente a aplicação.

2.6.1 Estratégia e execução de testes

A estratégia de testes do MotivaFit é integrada à arquitetura em camadas, permitindo a validação independente dos componentes e, depois, da integração entre eles. Os níveis (unitário, integração e sistema, conforme Pressman e Maxim, 2016) são os mesmos do Documento de Visão (seção 6.1) e são distribuídos conforme a Tabela 11.

Tabela 11 – Níveis de teste por camada

Nível	Foco arquitetural	Exemplos do roteiro
Unitário	Camada de Services do Backend (regras de treinos, registros, métricas e gamificação), com dependências isoladas ou simuladas; componentes e comportamentos relevantes da interface no Frontend.	Definidos junto ao código; não constam de T1 a T22
Integração	Controllers, Services e Repositories com o banco PostgreSQL, em ambiente controlado e reproduzível provisionado por contêineres Docker; integração das telas com os endpoints.	T1, T4, T5, T6, T8, T10, T11, T12, T13, T15, T17, T21, T22
Sistema	Fluxo completo do usuário, da interface (incluindo telas de celular) à resposta do servidor e à persistência; fluxos críticos podem ser validados por testes ponta a ponta.	T2, T3, T7, T9, T14, T16, T18, T19, T20

Fonte: elaborado pelos autores (2026), com base no Documento de Visão (GRUPO PERLMAN, 2026).

No Backend, os testes unitários focam a camada de Services, e a integração entre Controllers, Services e Repositories é validada por testes de integração contra o PostgreSQL. As ferramentas de teste serão definidas pela equipe de desenvolvimento (por exemplo, JUnit e Mockito no Backend e Jest ou Vitest com React Testing Library no Frontend). Os casos T1 a T22 estão planejados no Documento de Visão e seguem a regra Previsto = Realizado: qualquer divergência é tratada como defeito, corrigida e testada novamente até a aprovação.

2.6.2 Integração contínua

GitHub Actions executa builds e testes a cada abertura ou atualização de Pull Request. A Tabela 12 explicita os gatilhos: análise estática por Pull Request e varredura diária são execuções separadas. Após integração na branch principal, um job gera artefatos identificados pelo commit. A CI não constitui entrega contínua; deploy está fora da esteira descrita.

Tabela 12 – Etapas da pipeline de integração contínua

Etapa	Descrição	Bloqueante	Gatilho
1	Compilação do Backend (Maven) e build do Frontend.	Sim	Pull Request
2	Testes unitários do Backend e do Frontend.	Sim	Pull Request
3	Testes de integração com PostgreSQL provisionado em contêiner Docker.	Sim	Pull Request
4	Cobertura global ponderada M3 e por camada; meta ≥ 70%, gate após piloto documentado.	Após piloto	Pull Request
5	Análise estática por Pull Request; execução diária independente por agendamento.	Defeitos críticos ou Blocker	Pull Request + cron diário
6	Gerar imagens e build estático identificados pelo commit; armazenar artefatos. Não realiza deploy.	Falha impede artefato	Push na principal

Fonte: revisão proposta com base em M2, M3 e M4; a política de cobertura foi atualizada também na cópia do Documento de Visão 1.0.1.

2.7 Operação e ciclo de vida dos dados

Operação: esta seção concentra somente decisões operacionais e de ciclo de vida; tecnologias, acesso e padronização estão nas seções 2.3.3 e 2.4.2. Até a definição de hospedagem não há compromisso formal de disponibilidade. Requisitos de desempenho e usabilidade estão na seção 2.4.1.4.

Migrações: adotar scripts SQL versionados via Flyway no Backend. Testar migração de banco vazio e atualização da versão anterior na CI; aplicar migrações uma única vez antes de liberar uma nova versão, após backup. Não usar criação automática de schema pelo Hibernate em homologação ou produção. Preferir alterações compatíveis em etapas; rollback da aplicação exige schema compatível, e restauração do banco requer plano e validação próprios.

Backup e recuperação (plano para a primeira hospedagem): backup diário criptografado do PostgreSQL, fora do servidor principal, retenção de 30 dias e acesso restrito ao operador. Meta proposta RPO $\leq$ 24 h e RTO $\leq$ 4 h. Testar restauração antes da primeira publicação e mensalmente; registrar data, checksum e duração. Provedor e responsável devem ser identificados antes de operar. Não há evidência de backup implantado nesta revisão.

- Privacidade e LGPD: apresentar aviso de privacidade com finalidade, categorias de dados, bases legais, retenção e contato do controlador a serem definidos antes de dados reais. Coletar apenas o necessário; publicar resultados somente por ação explícita do titular. Dados de treino podem revelar saúde e exigem avaliação do tratamento de dados sensíveis (art. 11). Usar dados fictícios em testes, reduzir e proteger logs e limitar métricas administrativas a agregados. O direito de acesso, correção e eliminação é considerado nos termos dos arts. 16 e 18 da Lei 13.709/2018; a revisão não presume consentimento como base para todo tratamento.
- Exclusão de conta: DELETE /api/conta exige autenticação recente e confirmação explícita; bloqueia a conta e revoga tokens imediatamente. Um job elimina identificadores, rotinas, execuções, recordes, concessões e conteúdo pessoal em até 30 dias, ou documenta fundamento e prazo de retenção legal excepcional. Associações sociais são removidas e conteúdo de terceiros é preservado sem identificadores do titular. Logs de auditoria são minimizados e sujeitos a prazo definido. Backups expiram em até 30 dias; se restaurados antes disso, reaplicar solicitações de eliminação antes de reabrir acesso. GET /api/conta/exportacao permite obter os próprios dados. Esses fluxos transversais devem ser detalhados e testados no backlog técnico do MVP.
- Ciclo de vida: arquivamento de treino preserva histórico; exclusão de conta elimina dados pessoais conforme a política acima. Essa distinção deve ser visível na confirmação apresentada ao usuário e testada com treino contendo registros.

Decisões operacionais pendentes: escolha do provedor e operador, dimensionamento real, política de retenção por categoria e evidências de carga, restauração e cobertura. Os números apresentados são metas propostas, sem homologação ou ensaios realizados.

2.8 Rastreabilidade com o Documento de Visão

A Tabela 13 relaciona cada requisito do backlog ao módulo do Backend, às entidades envolvidas, à Sprint e à prioridade, garantindo que a arquitetura cubra todos os requisitos (RF-01 a RF-24).

Tabela 13 – Rastreabilidade entre requisitos, módulos e entidades

Requisito	Módulo	Entidades	Sprint	Prioridade
RF-01	Autenticação e Segurança	USUARIO	3	Must
RF-02	Autenticação e Segurança	USUARIO	3	Must
RF-03	Core de Treinos	TREINO	3	Must

Requisito	Módulo	Entidades	Sprint	Prioridade
RF-04	Core de Treinos	TREINO	3	Must
RF-05	Core de Treinos	TREINO	3	Must
RF-06	Core de Treinos	TREINO	3	Must
RF-07	Core de Treinos	EXERCICIO, CATALOGO_EXERCICIO	4	Must
RF-08	Core de Treinos	EXERCICIO	4	Must
RF-09	Core de Treinos	EXERCICIO	4	Must
RF-10	Core de Treinos	EXERCICIO	4	Must
RF-11	Registro e Métricas	REGISTRO_DIARIO	4	Must
RF-12	Registro e Métricas	REGISTRO_DIARIO	5	Should
RF-13	Autenticação e Segurança	CONVITE_ADMIN, USUARIO	5	Could
RF-14	Analítico e Admin	USUARIO, REGISTRO_DIARIO	5	Should
RF-15	Analítico e Admin	USUARIO, REGISTRO_DIARIO	6	Should
RF-16	Analítico e Admin	USUARIO, REGISTRO_DIARIO	6	Should
RF-17	Analítico e Admin	EXERCICIO, REGISTRO_DIARIO	7	Should
RF-18	Gamificação	DESAFIO, DESAFIO_ITEM, DESAFIO_PARTICIPANTE, AMIZADE	7	Should
RF-19	Gamificação	DESAFIO	7	Should
RF-20	Registro e Métricas	RECORDE_PESSOAL, REGISTRO_DIARIO, CATALOGO_EXERCICIO	8	Should
RF-21	Social	PUBLICACAO	8	Should
RF-22	Gamificação	CONQUISTA_DEFINICAO, USUARIO_CONQUISTA, REGISTRO_DIARIO	9	Should
RF-23	Social	GRUPO_TREINO, GRUPO_MEMBRO, CONVITE_GRUPO, PUBLICACAO, CURTIDA, COMENTARIO	9	Should
RF-24	Autenticação e Segurança	USUARIO	9 e 10	Could

Fonte: elaborado pelos autores (2026), com base no Documento de Visão (GRUPO PERLMAN, 2026).

2.9 Riscos arquiteturais e decisões pendentes

A Tabela 14 complementa o gerenciamento de riscos do Documento de Visão com os riscos de natureza arquitetural identificados nesta revisão.

Tabela 14 – Riscos arquiteturais e decisões pendentes

Risco / pendência	Mitigação arquitetural
Falha na integração com a autenticação Google (RF-24, Could have).	Isolar a autenticação externa no módulo de Autenticação e Segurança, mantendo o login por e-mail e senha sempre disponível (T13). Testar a requisição isoladamente via Postman antes de integrar ao Frontend e ao Backend.
Consultas do painel administrativo (RF-14 a RF-17) ficarem lentas com o crescimento do histórico.	Executar as agregações no banco, com índices por usuário e data nos registros diários, e avaliar paginação e cache caso a métrica de desempenho seja ultrapassada.

Risco / pendência	Mitigação arquitetural
Acoplamento entre a gamificação e o registro de exercícios.	Orquestrador chama interfaces de Registro e Gamificação; Gamificação consulta Registro, sem dependência inversa. Testes arquiteturais verificam ciclos.
Vazamento ou uso indevido de tokens JWT.	Tokens com expiração curta, chave de assinatura em variável de ambiente, comunicação exclusivamente por HTTPS e rejeição de tokens inválidos (T21 e T22).
Provedor de hospedagem ainda não definido.	Decidir antes da primeira implantação. A conteinerização com Docker mantém o sistema portável entre provedores.
Implementação do modelo social ainda não validada.	Implementar entidades e integridade da Figura 6; testar convite, adesão e privacidade antes da entrega.
Entregas semanais curtas podem atrasar requisitos Must.	Must limitado a RF-01 a RF-11; reavaliar Should e Could a cada Sprint. Mudanças de escopo devem atualizar Visão e Arquitetura.
Cobertura de 70% desde o primeiro Pull Request sem evidência.	Executar piloto na Sprint 3; registrar esforço e relatórios. Gate informativo até comprovação, com meta antes do fim do MVP.
Metas operacionais e legais ainda não homologadas.	Escolher operador/provedor; validar carga, restauração, retenção e aviso de privacidade antes de tratar dados reais.

Fonte: elaborado pelos autores (2026), com base no Documento de Visão (GRUPO PERLMAN, 2026).

3 BIBLIOGRAFIA

BASILI, Victor R.; CALDIERA, Gianluigi; ROMBACH, H. Dieter. The goal question metric approach. In: MARCINIAK, John J. (ed.). Encyclopedia of software engineering. New York: John Wiley & Sons, 1994. v. 1, p. 528-532.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 4. ed. Boston: Addison-Wesley, 2021.

FOWLER, Martin. Padrões de arquitetura de aplicações corporativas. Tradução de Francisco Araújo. Porto Alegre: Bookman, 2006.

GRUPO PERLMAN. Documento de Visão do Produto e do Projeto – MotivaFit. Versões 1.0.0 (original) e 1.0.1 (cópia revisada). Brasília: Universidade de Brasília, 2026.

KRUCHTEN, Philippe; NORD, Robert L.; OZKAYA, Ipek. Managing technical debt: reducing friction in software development. Boston: Addison-Wesley, 2019.

MARCZEWSKI, Andrzej. User Types. In: MARCZEWSKI, Andrzej. Even ninja monkeys like to play: gamification, game thinking and motivational design. 1. ed. [S.l.]: CreateSpace Independent Publishing Platform, 2015. p. 65–80.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SCHWABER, Ken; SUTHERLAND, Jeff. O Guia do Scrum: O Guia Definitivo para o Scrum: As Regras do Jogo. 2020.

TONDELLO, Gustavo F. et al. The gamification user types Hexad scale. In: ANNUAL SYMPOSIUM ON COMPUTER-HUMAN INTERACTION IN PLAY, 2016, Austin. Proceedings [...]. New York: ACM, 2016. p. 229-243. DOI: 10.1145/2967934.2968082.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (LGPD), arts. 11, 16 e 18. Texto compilado. https://planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 7 out. 2026.
MARCZEWSKI, Andrzej. Correcting the misconceptions around the Philanthropist user type and purpose. 2017. https://www.gamified.uk/2017/01/23/rediscovering-purpose-philanthropist-user-type/. Acesso em: 7 out. 2026.
TONDELLO, Gustavo F. et al. The gamification user types Hexad scale. 2016. Texto do artigo: https://www.gamefulbits.com/wp-content/uploads/2016/09/2016-The-Gamification-User-Types-Hexad-Scale.pdf. As associações no MotivaFit são hipóteses de projeto, não validação empírica.